HEURÍSTICA Y OPTIMIZACIÓN: PRACTICA 1



Nombre:	NIA:	Correo Electronico:	Grupo:	Grado:
Luis Enrique Valero Montero	100495982	100495982@alumnos.uc3m.es	84	Computer Science and Engineering
George Jose Hallak Pita	100459094	100459094@alumnos.uc3m.es	84	Computer Science and Engineering

TABLA DE CONTENIDOS

1.0 Introduccion..... 3
2.0 Modelos..... 3
2.1 Minimización del coste medio..... 3
Variables de decisión..... 3
Restricciones..... 4
Función objetivo..... 5
2.2 Montaje volumétrico..... 5
Variables de decisión..... 5
Restricciones..... 6
Función objetivo..... 7
2.3 Optimización de pallets incompletos..... 7
Variables de decision..... 7
Restricciones..... 8
Función objetivo..... 9
2.4 Optimización del montaje en una cantidad arbitraria de pallets incompletos..... 10
Variables de decision..... 10
Restricciones..... 11
Función objetivo..... 12
3.0 Análisis de resultados..... 13
3.1 Análisis de la Parte 1..... 13
3.2 Análisis de la Parte 2..... 13
3.3 Análisis de la Parte 3..... 14
3.4 Análisis de la Parte 4..... 14

1.0 Introduccion.

Esta memoria se trata como un documento técnico demostrando nuestro trabajo realizado para la Práctica 1 de Programación Lineal. El objetivo de esta práctica es aplicar la teoría que hemos visto sobre modelización de tareas de programación lineal a un problema del mundo real complejo para no solo saber modelizar sino también resolver estas tareas de forma óptima utilizando programas que hemos desarrollado.

El problema en términos simples se trata de una situación bastante cotidiana en logística: una compañía de logística tiene que colocar cada día cajas sobre pallets y quiere hacerlo de la mejor manera posible para reducir los costes de dicha operación. El problema se nos da en cuatro partes donde la primera parte es el problema lo más simplificado y con cada siguiente parte se le añade nueva información y más complejidad.

En la sección 2 de este documento se verá para cada de las partes de la práctica (parte 1-4): el "breakdown" o descripción del problema de programación lineal, las descripciones detalladas del modelo general formalizado con notaciones algebraicas escritas por LaTeX. Los modelos se describen en el orden que se nos ha enseñado en clase: variables de decisión, restricciones y después la función objetivo. Así ha sido nuestra forma de poder sacar cada modelo de programación lineal.

Para poder resolver las tareas óptimamente y encontrar soluciones óptimas para cada parte, se ha desarrollado un script de Python 3.12 que invoca el solver de GNU GLPK en el que dicho solver utiliza nuestro modelo diseñado escrito en el lenguaje MathProg (como archivo de modelo) y un archivo de datos del problema (generado por el mismo script de Python a partir de los datos de entrada encontrados en un fichero de input), donde al final la solución óptima se encuentra en un fichero de output y donde el número de variables de decisión creadas, el número de restricciones generadas y el valor óptimo de la función objetivo se imprimen en la pantalla.

En la sección 3 de este documento analizamos los resultados y el rendimiento de nuestros modelos de cada parte. Hablamos sobre la complejidad temporal de nuestra implementación y de cómo crecen los problemas mientras más crecen los parámetros y datos del problema. Vemos si existen casos infactibles y también si existen otras alternativas para poder resolver ciertos problemas óptimamente sin tener que utilizar la programación lineal.

2.0 Modelos.

2.1 Minimización del coste medio.

En esta primera parte hay que decidir dónde colocar $n = l^2$ cajas sobre un pallet cuadrangular de lado l, en un único nivel y llenando el pallet por completo. Se trata de un problema de Programación Lineal Entera 0-1, y más en concreto de un problema de asignación: cada caja tiene que ir a exactamente una posición y cada posición tiene que recibir exactamente una caja, con la condición añadida de respetar el orden de prioridades respecto al punto de acceso.

Tomamos como referencia la vista cenital del enunciado: la posición (1,1) es la esquina inferior izquierda, el primer índice indica la fila y el segundo la columna, y el punto de acceso está arriba, junto a la fila l. Por tanto, cuanto mayor es i, más cerca del acceso queda la caja.

Variables de decisión

Hemos usado una única familia de variables binarias, que representa la asignación de una caja a una posición específica en el pallet. La variable x[c,i,j] vale 1 si la caja c se coloca en la

fila i y la columna j, y 0 en caso contrario. Como hay l^2 cajas y l^2 posiciones, el modelo tiene $l^2 \cdot l^2 = l^4$ variables, que para un pallet de lado 3 son 81.

$$x_{c,i,j} \in \{0,1\}$$

$$c : identificador\ para\ cada\ caja, c \in \{1..n\},\ n = l^2$$

$$i : fila\, i - esima,\ j \in \{1..l\}$$

$$j : columna\, j - esima,\ j \in \{1..l\}$$

Restricciones

En total el modelo generará $n + l^2 + l \cdot (l-1)$ restricciones: n de la restricción 1, l^2 de la restricción 2 y $l \cdot (l-1)$ de la restricción 3. Para l = 3 son 24. La restricción 4 no está explícitamente escrita como una restricción dentro del modelo escrito en MathProg, si no que está aplicada en la declaración de la variable de decisión, por eso no se cuenta en el número de restricciones que se imprime al final de una resolución.

Restricción 1. Cada caja solo puede ocupar exactamente una posición del pallet: Para cada caja *c*, la suma de sus variables en cada combinación de fila y columna, es decir para cada respectiva posición del pallet debe ser 1, de forma que ninguna caja se quede fuera ni sea asignada a dos celdas distintas dentro del pallet.

$$\sum_{i=1}^{l}\sum_{j=1}^{l} x_{c,i,j} = 1 \qquad \forall\, c, 1 \le c \le n$$

Restricción 2. Cada posición (i,j) del pallet está ocupada por exactamente una caja: Como el número de cajas coincide con el de posiciones, esta restricción junto con la anterior garantiza que el pallet queda completamente lleno y que no hay dos cajas en el mismo sitio. Para cada combinación de *i* y *j* (cada posible posición), la suma de cada variable para cada una de las cajas respectiva a esa posición debe ser 1.

$$\sum_{c=1}^{n} x_{c,i,j} = 1 \qquad \forall\, i, 1 \le i \le l, , \quad \forall\, j, 1 \le j \le l$$

Restricción 3. Orden de prioridad respecto al punto de acceso: En cada columna j, las cajas se deben colocar de tal forma de que las de mayor prioridad (de menor valor p) estén más cerca al punto de acceso. En otras palabras, una caja tiene que tener por enfrente (respecto al punto de acceso) en la misma columna una caja con menor o igual valor p. Esto lo pudimos expresar de una forma en la que solo hace falta comparar 2 posiciones de una misma columna pero de 2 filas consecutivas de tal manera: dada una caja asignada a la posición (i,j), otra caja asignada enfrente en la posición (i+1,j) tendrá que tener un valor p menor o igual a la caja que tiene por detrás. Esta comparación se puede expresar como: para cada *i* de 1 a *l*-1 (como utilizamos el índice i+1 para identificar la caja por enfrente de otra, el rango llega hasta *l*-1) y cada j de 1 a *l*, la suma del producto entre el valor de prioridad y la variable de decisión de cada una de las cajas en (i+1,j), es menor o igual a los mismos términos pero para la posición (i,j). Como las restricciones 1 y 2 ya se aseguran de que todas las cajas son asignadas a una única celda y que ninguna celda esté vacía, la parte izquierda y derecha de esta restricción solo tendrá un valor no cero en cada lado ya que solo una variable x será 1 y el resto 0.

$$\sum_{c=1}^{n} p_c \cdot x_{c,i+1,j} \leq \sum_{c=1}^{n} p_c \cdot x_{c,i,j} \qquad \forall\, i, 1 \leq i \leq l-1, , \quad \forall\, j, 1 \leq j \leq l$$

Restricción 4. Naturaleza de las variables: todas son binarias, es decir que tienen el valor 0 o 1 para poder representar si la caja ha sido asignada o no a una respectiva celda.

$$x_{c,i,j} \in \{0,1\}$$

Función objetivo

El coste de una caja en la posición (i,j) es la suma de las prioridades de las cajas que tiene por delante en su columna, es decir, en las filas i+1 hasta *l*. Si en lugar de sumar caja a caja lo miramos al revés, la prioridad de la caja que está en la fila i aparece en el coste de todas las cajas que tiene detrás, que son las de las filas 1 hasta i−1. Por tanto, su prioridad se cuenta exactamente (i−1) veces. Esto nos permite escribir el coste total sin multiplicar variables entre sí, y el coste medio se obtiene dividiendo entre el número de cajas, n:

$$\min z = \frac{\sum_{c=1}^{n} \sum_{i=1}^{l} \sum_{j=1}^{l} (i-1) \cdot p_c \cdot x_{c,i,j}}{n}$$

La expresión es lineal, porque (i−1), p_c y *l* son datos del problema y no variables. Un detalle interesante es que la fila más cercana al acceso nunca aporta coste por sí misma, mientras que la del fondo es la que más pesa entonces el modelo tenderá por tanto a mandar al fondo las cajas con números de prioridad más altos, que es justo lo que pide la restricción 3.

2.2 Montaje volumétrico

En esta segunda parte se le ha añadido un poco más de complejidad al problema de la parte anterior, ahora el pallet puede tener un número arbitrario de niveles (*h*) y con esto cada caja ahora no solo tiene el atributo de prioridad pero también tiene un peso en gramos y una capacidad de soportar peso en gramos. Como menciona el enunciado deberíamos considerar las mismas restricciones y función objetivo pero adaptar el modelo para incluir la idea de un montaje volumétrico donde se pueden apilar las cajas. Los pallets siguen siendo cuadrangulares y se sigue teniendo que rellenar el pallet entero, es decir todos los niveles tendrán que llenarse. Por lo tanto se sigue tratando de un problema de programación lineal entera 0-1, específicamente de asignación.

Nuestro diseño para el modelo general de esta segunda parte viene siendo el mismo de la parte anterior, pero incluyendo una nueva dimensión para los niveles y añadiendo restricciones para considerar que las cajas deben ser asignadas de tal forma que puedan soportar el peso de todas las cajas que se encuentren apiladas directamente encima de ellas.

Variables de decisión

Seguimos con la misma variable de decisión que representa la asignación de una caja a una posición específica en el pallet. Simplemente que ahora le hemos añadido una nueva dimensión o índice para representar el nivel de la celda en el pallet. Por lo tanto una variable *x* tendrá el valor 1 si una caja *c* ha sido asignado la posición en fila *i*, columna *j* y nivel *k*, o 0 si esa caja no ha sido asignada esa exacta posición en el pallet. El número de variables que se crearan es $n \cdot l^2 \cdot h$ que es simplemente el producto entre todas las cardinalidades de cada índice de las variables.

$$x_{c,i,j,k} \in \{0,1\}$$

$$c : identificador\ para\ cada\ caja,\ c \in \{1..n\},\ n = hl^2$$

$$i : fila\ i-esima, i \in \{1..l\}$$

$$j : columna\ j-esima, j \in \{1..l\}$$

$$k : nivel\ k-esima, k \in \{1..h\}$$

Restricciones

En total el modelo generará $n + l^2 \cdot h + l\cdot(l-1)\cdot h + l^2\cdot(h-1)$ restricciones: *n* de la restricción 1, $l^2 \cdot h$ de la restricción 2, $l\cdot(l-1)\cdot h$ de la restricción 3 y $l^2\cdot(h-1)$ de la restricción 4. Para l = 2 y h = 3 son 38. La restricción 5 no está explícitamente escrita como una restricción dentro del modelo escrito en MathProg, si no que está aplicada en la declaración de la variable de decisión, por eso no se cuenta en el número de restricciones que se imprime al final de una resolución.

Restricción 1. Cada caja solo puede ocupar exactamente una posición del pallet: se sigue manteniendo la misma restricción que las partes anteriores solo que añadimos la dimensión de nivel. Dada una caja *c* la suma de todas las variables *x* para cada combinación de *i,j* y *k* tendrá que sumar a 1 para garantizar que no sea posible que la caja *c* esté en dos posiciones simultáneamente.

$$\sum_{i=1}^{l}\sum_{j=1}^{l}\sum_{k=1}^{h} x_{c,i,j,k} = 1 \qquad \forall c, 1 \le c \le n$$

Restricción 2. Cada posición (i,j,k) del pallet está ocupada por exactamente una caja: como antes solo añadimos la dimensión de nivel, dada una posición en el pallet la suma de todas las variables *x* para todas las cajas *c* con respeto a esa posición exacta debe sumar a 1.

$$\sum_{c=1}^{n} x_{c,i,j,k} = 1 \qquad \forall i, 1 \le i \le l,,\ \ \forall j, 1 \le j \le l,,\ \ \forall k, 1 \le k \le h$$

Restricción 3. Orden de prioridad respecto al punto de acceso para cada columna y nivel: la misma restricción de orden de prioridad como las partes anteriores introduciendo la dimensión de nivel. Para cada nivel *k* independientemente, se compara 2 filas consecutivas garantizando que la prioridad de caja que esté por enfrente (misma columna) con respeto al punta de acceso es de mayor prioridad (que el valor *p* sea menor en valor) y así se propaga para el resto de filas consecutivas en todos los niveles.

$$\sum_{c=1}^{n} p_c \cdot x_{c,i+1,j,k} \le \sum_{c=1}^{n} p_c \cdot x_{c,i,j,k} \qquad \forall i, 1 \le i \le l-1,,\ \ \forall j, 1 \le j \le l,,\ \ \forall k, 1 \le k \le h$$

Restricción 4. Capacidad: como se ha mencionado antes esta parte introduce la idea que hay niveles y que las caja se pueden apilar verticalmente con cada caja teniendo un peso y una capacidad de soportar un cierto peso. Para garantizar que la caja en una dada posición (*i,j,k*) pueda soportar las cajas por encima que serían los niveles *k+1* hasta *h-1* (porque las cajas del último nivel no soportan otras cajas) con la misma fila y columna, la capacidad de la caja c en un nivel *k,* debe ser mayor o igual a la suma total de los pesos de las cajas apiladas directamente encima.

$$\sum_{c=1}^{n} \gamma_c \cdot x_{c,i,j,k} \geq \sum_{c=1}^{n} \sum_{k'=k+1}^{h} \omega_c \cdot x_{c,i,j,k'} \qquad \forall i, 1 \leq i \leq l, , \ \forall j, 1 \leq j \leq l, , \ \forall k, 1 \leq k \leq h-1$$

Restricción 5. Naturaleza de las variables de decisión: garantiza que las variables representan una asignación o no solamente con los valores 1 y 0 respectivamente. Enforca que la variable sea binaria.

$$x_{c,i,j,k} \in \{0,1\}$$

Función objetivo

Como menciona el enunciado, el objetivo de la empresa sigue siendo de minimizar el coste medio de la disposición de todas las cajas y por lo cual sigue siendo la misma función objetivo que la primera parte pero ahora con la nueva dimensión de los niveles *k*.

$$\min z = \frac{\sum_{c=1}^{n} \sum_{i=1}^{l} \sum_{j=1}^{l} \sum_{k=1}^{h} (i-1) \cdot p_c \cdot x_{c,i,j,k}}{n}$$

2.3 Optimización de pallets incompletos

En esta tercera parte el caso es el mismo que en la parte anterior: pallets cuadrados de lado l, cajas con prioridad, peso y capacidad, y apilamiento en varios niveles. La diferencia es que ahora el número de cajas, *n*, puede ser menor que el número de posiciones del pallet: $1 \leq n \leq h \cdot l^2$, es decir no es estrictamente necesario rellenar el pallet entero, se permiten huecos o celdas vacias. Por tanto, en la solución óptima puede haber posiciones vacías. El enunciado da tres reglas para ellas. No se puede poner una caja encima de una posición vacía, porque su capacidad es 0. La prioridad de una posición vacía es +∞, es decir, la más baja posible. Y el coste de una posición vacía es 0. Sigue siendo un problema de Programación Lineal Entera, aunque ahora es mixta, porque además de las variables binarias de asignación hay variables de coste. Ya no es un problema de asignación perfecta como en las partes 1 y 2, porque cada posición puede quedarse vacía. Por eso hay que cambiar la restricción de ocupación de las posiciones, adaptar la de prioridades para que tenga en cuenta los huecos y añadir una restricción nueva que impida apilar cajas sobre ellos. Además, como explicamos en la función objetivo, el número de cajas que hay detrás de cada posición ya no es fijo. Así que el coste ya no se puede calcular directamente a partir de x y hace falta una nueva familia de variables.

Variables de decision

Usamos la misma variable de asignación que en la parte 2, con índices de caja, fila, columna y nivel. Además, añadimos una familia nueva de variables para el coste de cada posición. x[c,i,j,k] vale 1 si la caja c va en la fila i, la columna j y el nivel k, y 0 si no. Si la suma de x[c,i,j,k] sobre todas las cajas vale 0, la posición (i,j,k) está vacía, y si vale 1, está ocupada. Esa suma nos sirve como indicador de ocupación sin tener que definir otra variable.

$$x_{c,i,j,k} \in \{0,1\}$$

$$c : identificador\ para\ cada\ caja, c \in \{1..n\}, n \leq hl^2$$

$$i : fila\ i-esima, i \in \{1..l\}$$

$$j : columna\ j-esima, j \in \{1..l\}$$

$$k : nivel\ k-esima, k \in \{1..h\}$$

La variable u[i,j,k] es el coste de la posición (i,j,k). Si está ocupada, es la suma de las prioridades de las cajas que tiene delante en su misma columna y nivel. Si está vacía, vale 0. La fila l, que es la más cercana al acceso, nunca tiene cajas delante, así que su coste siempre es 0 y no le hace falta variable. Por eso el índice i solo va de 1 a $l - 1$.

$$u_{i,j,k} \in \mathbb{N}_0$$
$$i : fila\ i - esima, i \in \{1..l-1\}$$
$$j : columna\ j - esima, j \in \{1..l\}$$
$$k : nivel\ k - esima, k \in \{1..h\}$$

Restricciones

En total el modelo generará $n + l^2 \cdot h + l\cdot(l-1)\cdot h + l^2\cdot(h-1) + l^2\cdot(h-1) + l\cdot(l-1)\cdot h$ restricciones: n de la restricción 1, $l^2 \cdot h$ de la restricción 2, $l\cdot(l-1)\cdot h$ de la restricción 3 y 6, $l^2\cdot(h-1)$ de la restricción 4 y 5, . Para $l = 2$, h = 3 y n = 9 son 49. La restricción 7 y 8 no están explícitamente escrita como una restricción dentro del modelo escrito en MathProg, si no que están aplicadas en la declaración de las variables de decisión, por eso no se cuenta en el número de restricciones que se imprime al final de una resolución.

Restricción 1. Cada caja solo puede ocupar exactamente una posición del pallet: Misma restricción que en la parte 2.

$$\sum_{i=1}^{l}\sum_{j=1}^{l}\sum_{k=1}^{h} x_{c,i,j,k} = 1 \qquad \forall c, 1 \le c \le n$$

Restricción 2. Cada posición del pallet contiene como máximo una sola caja, pero puede quedar vacía: En la parte 2 cada posición tenía que tener exactamente una caja. Ahora la suma de las x de cada posición tiene que ser menor o igual que 1, y eso permite que el pallet quede incompleto cuando n < h·l².

$$\sum_{c=1}^{n} x_{c,i,j,k} \le 1 \qquad \forall i, 1 \le i \le l, , \ \forall j, 1 \le j \le l, , \ \forall k, 1 \le k \le h$$

Restricción 3. Orden de prioridad respecto al punto de acceso para cada columna y nivel: En cada columna j y nivel k, la posición de la fila i+1, que está más cerca del acceso, tiene que tener una prioridad igual o mayor (un número igual o menor) que la de la fila i. Como una posición vacía tiene prioridad +∞, la representamos con p_max + 1, que es mayor que cualquier prioridad real. La prioridad de cada posición se escribe como la suma de p_c·x[c,i,j,k] más (p_max + 1)·(1 − Σ x[c,i,j,k]). Si la posición está ocupada, el segundo término vale 0 y queda la prioridad de la caja. Si está vacía, el primero vale 0 y queda p_max + 1. Así ninguna caja puede quedar detrás de un hueco, y los huecos siempre quedan al fondo, lejos del acceso.

$$(\sum_{c=1}^{n} p_c \cdot x_{c,i+1,j,k} + (p_{max}+1)\cdot(1 - \sum_{c=1}^{n} x_{c,i+1,j,k})) \le (\sum_{c=1}^{n} p_c \cdot x_{c,i,j,k} + (p_{max}+1)\cdot(1 - \sum_{c=1}^{n} x_{c,i,j,k}))$$
$$\forall i, 1 \le i \le l-1, , \ \forall j, 1 \le j \le l, , \ \forall k, 1 \le k \le h$$

Restricción 4. Capacidad: la caja que está en la posición (i,j,k) tiene que aguantar el peso de todas las cajas que tiene encima, en los niveles k+1 a h de la misma fila y columna. Si la posición está vacía, el lado izquierdo vale 0.

$$\sum_{c=1}^{n} \gamma_c \cdot x_{c,i,j,k} \geq \sum_{c=1}^{n} \sum_{k'=k+1}^{h} \omega_c \cdot x_{c,i,j,k'} \qquad \forall i, 1 \leq i \leq l, , \ \forall j, 1 \leq j \leq l, , \ \forall k, 1 \leq k \leq h-1$$

Restricción 5. No se puede colocar cajas sobre una posición que se encuentre vacía en el pallet: La ocupación de la posición (i,j,k+1) no puede ser mayor que la de la posición (i,j,k) que tiene debajo. Como las dos ocupaciones valen 0 o 1, no hace falta ningún coeficiente más. La restricción 4 ya impide poner cajas con peso sobre un hueco, pero esta sigue siendo necesaria para las cajas de peso 0, que la restricción 4 dejaría pasar.

$$\sum_{c=1}^{n} x_{c,i,j,k+1} \leq \sum_{c=1}^{n} x_{c,i,j,k} \qquad \forall i, 1 \leq i \leq l, , \ \forall j, 1 \leq j \leq l, , \ \forall k, 1 \leq k \leq h-1$$

Restricción 6. Coste de cada posición: Si la posición (i,j,k) está ocupada, u[i,j,k] tiene que ser como mínimo la suma de las prioridades de las cajas que tiene delante, en las filas i+1 a l de la misma columna y nivel. Si está vacía, el término (l − i)·p_max hace que el lado derecho sea menor o igual que 0. Ese término es una cota superior de la suma, porque delante hay como mucho l − i cajas y cada una tiene prioridad como mucho p_max. La restricción se relaja y u puede valer 0. Es una restricción de tipo "Big-M" (o también conocido como el método de la M mayúscula como vimos en clase), y hemos ajustado la constante al menor valor posible para que la relajación lineal sea lo más fuerte posible. Como la función objetivo minimiza la suma de las u, en el óptimo cada u vale exactamente el coste de su posición.

$$u_{i,j,k} \geq \sum_{c=1}^{n} \sum_{i'=i+1}^{l} p_c \cdot x_{c,i',j,k} - (l-i) \cdot p_{max} \cdot (1 - \sum_{c=1}^{n} x_{c,i,j,k}) \qquad \forall i, 1 \leq i \leq l-1, , \ \forall j, 1 \leq j \leq l, , \ \forall k, 1 \leq k \leq h$$

Restricción 7. Naturaleza de las variables de decisión: garantiza que las variables representan una asignación o no solamente con los valores 1 y 0 respectivamente. Enforca que la variable sea binaria.

$$x_{c,i,j,k} \in \{0,1\}$$

Restricción 8. Naturaleza de las variables de decisión: esta variable de decisión que representa el coste de una posición tiene que ser estrictamente 0 si la posición está vacía o un entero positivo para representar el coste de la caja en esa posición (i,j,k).

$$u_{i,j,k} \in \mathbb{N}_0$$

Función objetivo

Se minimiza el coste medio de la disposición, es decir, la suma de los costes de todas las posiciones dividida entre el número de cajas, n. El coste de cada posición es su variable u, que, como se explica en la restricción 6, vale exactamente el coste de la caja si la posición está ocupada y 0 si está vacía, tal como pide el enunciado.

$$\min z = \frac{\sum_{i=1}^{l-1} \sum_{j=1}^{l} \sum_{k=1}^{h} u_{i,j,k}}{n}$$

La expresión es lineal porque *n* es un dato del problema. Aquí no podemos usar el truco de la parte 1 de contar cada prioridad (i − 1) veces, porque eso da por hecho que todas las posiciones detrás de una caja están ocupadas. Con huecos, el número de cajas que hay detrás de cada una depende de la solución, y calcularlo directamente a partir de x obligaría a multiplicar variables entre sí. Con las variables u y la restricción 6 podemos expresar el coste sin ningún producto de variables.

2.4 Optimización del montaje en una cantidad arbitraria de pallets incompletos

Variables de decision

Como todas las partes anteriores seguimos con la misma variable de decisión para representar las asignaciones de cajas a posiciones dentro del pallet, simplemente que ahora le añadimos una nueva dimensión o índice para identificar el pallet como ahora hay que considerar la existencia de una disponibilidad ilimitada de pallets.

El rango de posibles valores para la m es de 1 a *M* donde *M* es el techo del número de cajas dividido entre la longitud del pallet. Al principio decíamos que el número de pallets está acotado por arriba por el número de cajas, y esto sigue siendo correcto. Esto sería el valor máximo absoluto para la cantidad de pallets que se pueden usar que sería asignar una caja por pallet, aunque sea válido no viene siendo lo mas optimo. Nos dimos cuenta que el número de pallets que realmente se necesitara óptimamente (donde las cajas no aportaran al coste si no solo los coste de los pallets que se usan) está acotado por arriba por el número de filas más cercanos al punto de acceso para cada nivel. Esto se define como el número de cajas (*n*) que hay que montar dividido por el número de columnas (*l*) que hay, que esto viene siendo el número de grupos de filas de índice *l* que se pueden formar para colocar todas las cajas en lo más cercano al punto de acceso. Como esta división puede formar un número que no sea entero, para garantizar que sea entero se toma el techo de esta división.

$$x_{c,i,j,k,m} \in \{0,1\}$$

$$c : identificador\ para\ cada\ caja, c \in \{1..n\}, n \leq hl^2$$

$$i : fila\ i-esima, i \in \{1..l\}$$

$$j : columna\ j-esima, j \in \{1..l\}$$

$$k : nivel\ k-esima, k \in \{1..h\}$$

$$m : identificador\ de\ pallet\ (pallet\ m-esima), m \in \{1..M\}, M = \left\lceil \frac{n}{l} \right\rceil$$

La variable u[i,j,k,m] es el coste de la posición (i,j,k,m). Si está ocupada, es la suma de las prioridades de las cajas que tiene delante en su misma columna y nivel y del mismo pallet m. Si está vacía, vale 0. La fila l, que es la más cercana al acceso, nunca tiene cajas delante, así que su coste siempre es 0 y no le hace falta variable. Por eso el índice i solo va de 1 a $l-1$.

$$u_{i,j,k,m} \in \mathbb{N}_0$$

$$i : fila\ i-esima, i \in \{1..l-1\}$$

$$j : columna\ j-esima, j \in \{1..l\}$$

$$k : nivel\ k-esima, k \in \{1..h\}$$

$$m : identificador\ de\ pallet\ (pallet\ m-esima), m \in \{1..M\}, M = \left\lceil \frac{n}{l} \right\rceil$$

La variable z[m] vale 1 si el pallet m se monta, es decir, si contiene al menos una caja, y 0 en caso contrario. Esta variable es imprescindible para poder escribir el coste de los pallets de forma lineal. El coste P solo debe pagarse por los pallets que contienen alguna caja, así que el término que queremos sumar es P multiplicado por el número de pallets no vacíos. Contar si un pallet está vacío o no a partir de las variables x es una función no lineal (vale 1 en cuanto la suma de sus x es mayor que 0, sin importar cuántas cajas tenga). La variable z[m] resuelve este problema: la restricción 2 obliga a que z[m] valga 1 en cuanto el pallet m contiene alguna caja, y

como la función objetivo minimiza P·Σ z[m], si el pallet está vacío el solver le dará el valor 0. Así, Σ z[m] es exactamente el número de pallets montados.

$$z_m \in \{0,1\}$$

$$m : identificador\ de\ pallet\ (pallet\ m-esima), m \in \{1..M\}, M = \left\lceil \frac{n}{l} \right\rceil$$

Restricciones

El modelo tiene en total Σ_c min(c, M)·h·l² variables x, M variables z y (l − 1)·l·h·M variables u. En cuanto a las restricciones, hay n de la restricción 1. Por cada pallet hay h·l² de la 2, h·l·(l − 1) de la 3, l²·(h − 1) de la 4, l²·(h − 1) de la 5 y h·l·(l − 1) de la 6. A eso hay que sumarle las M − 1 de la restricción 7. En total salen n + M·[h·l² + 2l²(h − 1) + 2h·l(l − 1)] + (M − 1).

Restricción 1. Cada caja solo puede ocupar exactamente una posición del pallet: Para cada caja c, la suma de sus variables en todos los pallets, filas, columnas y niveles tiene que ser 1. Así no se queda ninguna caja fuera y ninguna acaba en dos sitios, ni en el mismo pallet ni en pallets distintos.

$$\sum_{m=1}^{M}\sum_{i=1}^{l}\sum_{j=1}^{l}\sum_{k=1}^{h} x_{c,i,j,k,m} = 1 \qquad \forall c, 1 \le c \le n$$

Restricción 2. Cada posición del pallet contiene como máximo una sola caja, pero puede quedar vacía: En cada posición (i,j,k) de cada pallet m, la suma de las x de todas las cajas no puede pasar de z[m]. Esta restricción hace dos cosas a la vez. Si el pallet se monta (z[m] = 1), es la misma que en la parte 3

$$\sum_{c=1}^{n} x_{c,i,j,k,m} \le z_m \qquad \forall i, 1 \le i \le l, ,\ \forall j, 1 \le j \le l, ,\ \forall k, 1 \le k \le h, ,\ \forall m, 1 \le m \le M$$

Restricción 3. Orden de prioridad respecto al punto de acceso para cada columna y nivel: Es la misma restricción que en la parte 3, pero ahora se aplica a cada pallet por separado.

$$(\sum_{c=1}^{n} p_c \cdot x_{c,i+1,j,k,m} + (p_{max}+1)\cdot(1 - \sum_{c=1}^{n} x_{c,i+1,j,k,m})) \le (\sum_{c=1}^{n} p_c \cdot x_{c,i,j,k,m} + (p_{max}+1)\cdot(1 - \sum_{c=1}^{n} x_{c,i,j,k,m}))$$

$$\forall i, 1 \le i \le l-1, ,\ \forall j, 1 \le j \le l, ,\ \forall k, 1 \le k \le h, ,\ \forall m, 1 \le m \le M$$

Restricción 4. Capacidad: La caja que está en la posición (i,j,k) del pallet m tiene que aguantar el peso de todas las cajas que tiene encima, en los niveles k+1 a h de la misma fila, columna y pallet.

$$\sum_{c=1}^{n} \gamma_c \cdot x_{c,i,j,k,m} \ge \sum_{k'=k+1}^{h}\sum_{c=1}^{n} \omega_c \cdot x_{c,i,j,k',m}$$

$$\forall i, 1 \le i \le l, ,\ \forall j, 1 \le j \le l, ,\ \forall k, 1 \le k \le h-1, ,\ \forall m, 1 \le m \le M$$

Restricción 5. No se puede colocar cajas sobre una posición que se encuentre vacía en el pallet: en cada pallet, la ocupación de la posición (i,j,k+1) no puede ser mayor que la de la posición (i,j,k) que tiene debajo. Como las dos ocupaciones valen 0 o 1, no hace falta ningún coeficiente más.

$$\sum_{c=1}^{n} x_{c,i,j,k+1,m} \le \sum_{c=1}^{n} x_{c,i,j,k,m} \qquad \forall i, 1 \le i \le l, ,\ \forall j, 1 \le j \le l, ,\ \forall k, 1 \le k \le h-1, ,\ \forall m, 1 \le m \le M$$

Restricción 6. Coste de cada posición: Si la posición (i,j,k) del pallet m está ocupada, u[i,j,k,m] tiene que ser como mínimo la suma de las prioridades de las cajas que tiene delante, en las filas i+1 a l de la misma columna, nivel y pallet. Si está vacía, el término (l − i)·p_max, que es una cota superior de esa suma, relaja la restricción y u puede valer 0.

$$u_{i,j,k,m} \geq \sum_{i'=i+1}^{l} \sum_{c=1}^{n} p_c \cdot x_{c,i',j,k,m} - (l-i) \cdot p_{max} \cdot (1 - \sum_{c=1}^{n} x_{c,i,j,k,m})$$

$$\forall i, 1 \leq i \leq l-1, , \ \forall j, 1 \leq j \leq l, , \ \forall k, 1 \leq k \leq h, , \ \forall m, 1 \leq m \leq M$$

Restriccion 7. Ruptura de simetrias: Un pallet solo se puede montar si se ha montado el anterior. Así los pallets usados siempre son los primeros (1, 2, …, K) y el solver no pierde tiempo con soluciones equivalentes que solo cambian qué pallets se quedan vacíos. Esta restricción encaja con la numeración de los pallets según su menor caja que explicamos en las variables de decisión.

$$z_{m+1} \leq z_m \qquad \forall m, 1 \leq m \leq M-1$$

Restricción 8. Naturaleza de las variables de decisión: garantiza que las variables representan una asignación o no solamente con los valores 1 y 0 respectivamente. Enforca que la variable sea binaria.

$$x_{c,i,j,k,m} \in \{0,1\}$$

Restricción 9. Naturaleza de las variables de decisión: esta variable de decisión que representa el coste de una posición tiene que ser estrictamente 0 si la posición está vacía o un entero positivo para representar el coste de la caja en esa posición (i,j,k,m).

$$u_{i,j,k,m} \in \mathbb{N}_0$$

Restricción 10. Naturaleza de las variables de decisión: la variable z[m] vale 1 si el pallet m se monta, es decir, si contiene al menos una caja, y 0 en caso contrario. Hay que asegurar que la variable sea binaria.

$$z_m \in \{0,1\}$$

Función objetivo

El coste total es el coste de disposición de todas las cajas, calculado en cada pallet por separado, más el coste de montar un pallet P por cada pallet montado. El primer término es la suma de todas las u, que, como se explica en la restricción 6, valen exactamente el coste de cada posición ocupada y 0 en las vacías. El segundo es P (coste de montar o usar un nuevo pallet) por el número de pallets montados, que es la suma de las z. Los dos términos son lineales porque P es un dato del problema.

$$\min z = \sum_{i=1}^{l-1} \sum_{j=1}^{l} \sum_{k=1}^{h} \sum_{m=1}^{M} u_{i,j,k,m} + P_{coste\ pallet} \cdot \sum_{m=1}^{M} z_m$$

Aquí no dividimos entre n como en las partes anteriores, porque el enunciado pide el coste total y no el medio. Tampoco podemos usar el truco de la parte 1 de contar cada prioridad (i − 1) veces: con posiciones vacías, el número de cajas que hay detrás de cada una ya no es fijo. Por eso seguimos usando las variables u como en la parte 3.

3.0 Análisis de resultados.

3.1 Análisis de la Parte 1

Para la parte 1 se probaron cuatro tipos de entradas para las prioridades: Aleatorias, todas distintas, muchos empates y todos iguales, con $l = 1$ hasta $l = 25$ que marcaba un límite. Lo que pudimos observar es que el tamaño crece muy deprisa, por tener l^4 variables binarias (n=l^2 cajas x l^2 posiciones), pero sólo crecían O(l^2) las restricciones, lo que hace un modelo muy ancho y poco alto. Ya que todo termina resolviendose en el nodo raíz, la mayoría del tiempo se consume en el mismo simplex inicial (el cual empieza a dispararse a partir de l=10 que da un tmedio=222s). También hay que tomar en cuenta que con las pruebas en las que habían muchos empates de prioridades o con todas iguales, se resuelve de 3 a 15 veces más rápido, ya que el preprocesador y el simplex encuentran muchas soluciones equivalentes, mientras que con todas las prioridades distintas sería el caso más lento.

Podríamos resolverlo sin programación lineal ordenando las cajas con sus prioridades, primero la de mayor prioridad en el primer espacio de la primera columna, y apilar sobre esta las cajas consecutivas hasta cumplir una columna de l cajas, así sucesivamente hasta consumir todas las cajas.

No existen instancias infactibles porque n = l^2, lo que garantiza una disposición ordenada que cumpla todas las restricciones (siempre y cuando sean valores enteros el número de cajas). También es acotada, ya que existe un número finito de soluciones binarias y el objetivo siempre es mayor a cero.

Puede haber varios valores óptimos para el mismo planteamiento del problema, ya que podrías permutar cajas entre columnas de una misma fila sin afectar el resultado de la función objetivo, lo que garantiza que tendrán el mismo coste óptimo.

3.2 Análisis de la Parte 2

Siguiendo con el análisis de resultados para la parte 2, hay que rescatar que las capacidades convierten el problema planteado en la parte 1, que era trivial, en un problema "difícil". El modelo crece ahora de esta manera:

- Variables: $n \cdot l^2 \cdot h = h^2 \cdot l^4$ ($n = h \cdot l^2$ cajas por $h \cdot l^2$ posiciones).
- Restricciones: $n + h \cdot l^2 + h \cdot l(l-1) + l^2(h-1) = 4hl^2 - hl - l^2$.

Con lo cual, aumentar h pesa mucho más que aumentar l.

Lo que notamos en las pruebas es lo siguiente:

- Con capacidades holgadas se comporta como en la parte 1, la solución parte del nodo raiz y resuelve aproximadamente 190.000 variables en 22s.
- Con capacidades que de verdad restringen, el límite baja a cientos o unos pocos miles de variables, con 3x3 GLPK en algunos casos no muestra un resultado óptimo en 2 minutos y que incluso agota su tiempo.
- Predomina lo ajustado de la capacidad más que el tamaño, un 4x2 con capacidad ajustada tarda 0.07s mientras que un 3x3 se queda sin tiempo. Cuando está ajustado la relajación lineal es débil y tiene que explorar decenas de miles de nodos.

Podría resolverse sin programación lineal para casos particulares, con h=1 o con capacidades que nunca se activan sirve la ordenación de la parte 1. Sin embargo, no hay un método directo equivalente que pueda garantizar resolverse sin programación lineal para un caso general.

En este caso a diferencia de la primera parte, si hay instancias infactibles por dos causas: La primera causa: hay más de l^2 cajas que no aguantan ni la caja más ligera. GLPK lo detecta enseguida en el preproceso para instancias 1*4, 2*6 y 3*4. La segunda causa: Las capacidades por sí solas tienen solución pero no son compatibles con el orden de prioridades. También es bueno aportar que, cuando tiene solución, es acotada siempre, tiene un número finito de soluciones y la función objetivo siempre es mayor a 0.

Pensando en si pueden haber varios valores óptimos, la respuesta es sí, porque puedes intercambiar columnas enteras (toda su fila y todos sus niveles) con otra columna mantiene el coste y todas las restricciones, y con las capacidades holgadas puedes permutar libremente cajas dentro de una fila y entre niveles.

3.3 Análisis de la Parte 3

Con la parte 3, realizamos 45 instancias variando l, h, ocupación y capacidad, gracias a esto podemos decir que el modelo es correcto pero la formulación que usa "Big-M" hace que GLPK tarde mucho en demostrar el óptimo, incluso con cientos de variables. También notamos que el modelo crece de la siguiente manera:

- Variables: $n \cdot h \cdot l^2$ binarias x más $(l-1) \cdot l \cdot h$ continuas u. Ahora dependen de n, no solo de l y h.
- Restricciones: $n + hl^2 + 2l^2(h-1) + 2hl(l-1)$.

Tras realizar las pruebas, podemos determinar que el límite práctico baja a unas 200-300 variables, frente a las miles de las parte 2 y las más de 100.000 de la parte 1. Usando capacidades holgadas se queda sin tiempo, logra conseguir el óptimo pero tarda en demostrar, causado por "Big-M", desactivando restricciones por posiciones "medio ocupadas". También hay que destacar que la ocupación influye mucho, si dejamos 100% de restricciones a "como mucho una caja" se vuelve la parte 2, pero entre 50% y 75% es más difícil, ya que hay muchas formas equivalentes de colocar huecos.

Se podría resolver sin programación lineal si las capacidades no limitan, ya que te aseguras que no queden cajas sobre espacios vacíos. Si $n \leq l^2$, se podrían poner todas las cajas en el nivel 1, aunque no garantiza que sea la solución óptima, ya que apilar cajas en la fila de acceso puede bajar el coste. Sin embargo, con capacidades que limitan no hay un método directo para resolver el problema.

Para este caso pueden haber soluciones infactibles, aunque menos que en la parte 2, basados en que si $n > l^2$ y hay más de l^2 cajas que no aguantan ni la caja más ligera, no habría solución, además, cuando esto no sucede, hay soluciones finitas acotadas por las restricciones cuando se minimiza.

Destacamos el hecho de que al igual que en la parte 2, hay varios óptimos e incluso mucho más, ya que además de poder intercambiar columnas enteras, podrías variar donde van los huecos.

3.4 Análisis de la Parte 4

Finalizando nuestro análisis de resultados, podemos determinar que el problema crece de la siguiente manera, tomando en cuenta $M = \lceil n/l \rceil$ pallets candidatos:

- Variables: $\Sigma_c \min(c, M) \cdot h \cdot l^2$ variables x, más M variables z, más $(l-1) \cdot l \cdot h \cdot M$ variables u.
- Restricciones: $n + M \cdot [h \cdot l^2 + 2l^2(h-1) + 2h \cdot l(l-1)] + (M-1)$.

Por ejemplo, para el caso del enunciado ($n = 6$, $l = 2$, $h = 2$, $M = 3$) son 135 variables y 80 restricciones, y para los datos de la parte 3 con $P = 1$ ($n = 9$, $l = 2$, $h = 3$, $M = 5$) son 455 variables y 213 restricciones.

Con las pruebas determinamos que el límite práctico está entre 500 y 1200 variables. Es un límite más alto que el de la parte 3, aunque el modelo sea esencialmente M copias de aquel, porque en esta parte el script invoca GLPK con las opciones --cuts y --pcost: sin ellas, el caso de 455 variables tarda unos 45 segundos, y con ellas unos 2. También observamos que las capacidades aleatorias suelen costar más que las holgadas, aunque no siempre, como en el caso 2x3 con $n = 9$, donde ocurre al revés.

Puede resolverse sin programación lineal en algunos casos particulares. Con $P = 0$, basta con poner l cajas por pallet, todas en la fila del acceso del nivel 1, lo que da coste 0, que es el mínimo posible. Con $n \leq l$, basta un solo pallet con todas las cajas en la fila del acceso del nivel 1, con coste P. Con capacidades holgadas también hay una regla directa: para cada número de pallets K posible, se ordenan las cajas por prioridad y se reparten llenando primero las filas del acceso de todos los pallets y niveles, después la segunda fila, y así sucesivamente, como en la parte 1; al final se elige el K con menor coste total. Sin embargo, con capacidades que limitan no hay una regla directa para resolver el problema sin programación lineal.

A diferencia de las partes 2 y 3, en esta parte nunca hay instancias infactibles: como se explicó en el apartado 2.4, colocar l cajas en cada pallet, en la fila del acceso del nivel 1, es siempre una solución válida, sean cuales sean los pesos y las capacidades. Tampoco puede haber soluciones no acotadas, porque todos los costes son mayores o iguales que 0. Además, suele haber varias soluciones óptimas: por cómo se reparten las cajas entre los pallets (como en el ejemplo del enunciado, donde nuestra solución es distinta a la de la Figura 4 con el mismo coste), por la simetría entre pallets, cuando $P = 0$, o cuando montar un pallet más cuesta exactamente lo mismo que lo que se ahorra en coste de colocación.